

Timesheet Management System – Project Specification

1 . Project Overview

A web-based Timesheet Management System that allows employees to log daily work hours and submit weekly timesheets for manager approval. Managers review, approve, or reject submissions. The system provides visibility into employee work hours and generates reports.

Primary goals: - Track employee work hours - Manage weekly timesheet submissions - Provide approval workflow - Generate time tracking reports

The system is NOT a full HRMS. It focuses only on time tracking.

2 . Technology Stack

Backend

- Java
- Spring Boot
- Spring Security (JWT authentication)
- JPA / Hibernate
- REST APIs

Frontend

- React
- TailwindCSS
- shadcn/ui component library
- React Query (TanStack Query)
- Axios for API calls

Database

- PostgreSQL or MySQL
-

3 . User Roles

Admin

Responsibilities: - Manage employees - Manage projects - Manage holiday calendar - View all reports

Manager

Responsibilities: - View team members - Review submitted timesheets - Approve or reject timesheets

Employee

Responsibilities: - Create timesheet entries - Submit weekly timesheets - View own history

4 . Core Modules

4 . 1 Authentication Module

Features: - Login - Logout - JWT token authentication - Role-based authorization

Pages: - Login Page

API Endpoints: - POST /auth/login - POST /auth/logout

4 . 2 Employee Management Module

Admin can manage employees.

Features: - Create employee - Edit employee - Activate / deactivate employee - Assign manager

Employee Fields: - id - name - email - role - department - manager_id

Pages: - Employee List - Add Employee - Edit Employee

API Endpoints: - GET /employees - POST /employees - PUT /employees/{id} - DELETE /employees/{id}

4 . 3 Project Management Module

Projects represent the work employees log hours against.

Features: - Create project - Update project - Assign employees to projects

Project Fields: - id - name - description - status

API Endpoints: - GET /projects - POST /projects - PUT /projects/{id}

5 . Timesheet Module

5 . 1 Daily Timesheet Entry

Employees log their work daily.

Fields: - date - project - task description - hours worked

Features: - Add entry - Edit entry - Delete entry

Rules: - Max 24 hours per day - Cannot log time on future dates

API: - GET /timesheets/daily - POST /timesheets - PUT /timesheets/{id} - DELETE /timesheets/{id}

5 . 2 Weekly Timesheet

Timesheets are submitted weekly.

Features: - View weekly entries - Automatic weekly hour calculation - Submit timesheet

Status Flow:

Draft -> Submitted -> Approved Draft -> Submitted -> Rejected

API: - GET /timesheets/week - POST /timesheets/submit

6 . Timesheet Approval Module

Managers review submitted timesheets.

Features: - View pending approvals - Approve timesheet - Reject timesheet - Add rejection comment

API: - GET /approvals/pending - POST /approvals/{id}/approve - POST /approvals/{id}/reject

7 . Holiday Management Module

Admin manages company holidays.

Features: - Add holiday - Delete holiday - View holiday calendar

Rules: - Employees cannot log hours on holidays

Fields: - id - holiday_name - holiday_date

API: - GET /holidays - POST /holidays - DELETE /holidays/{id}

8 . Dashboard Module

Employee Dashboard

Displays: - Weekly hours logged - Timesheet submission status - Pending submissions

Manager Dashboard

Displays: - Pending approvals - Team weekly hours summary

9 . Reporting Module

Reports include:

Employee Timesheet Report

Shows employee work hours over a period.

Project Hours Report

Shows hours logged per project.

Weekly Summary Report

Shows weekly totals per employee.

Features: - Date filtering - Export CSV

API: - GET /reports/employee - GET /reports/project

10 . Notifications (Optional)

Possible notifications:

- Reminder to submit weekly timesheet
 - Notification when approved
 - Notification when rejected
-

1 1 . Database Tables (High Level)

Tables:

Users Projects Timesheets TimesheetEntries Holidays Approvals

Relationships:

User -> Timesheet (1:N) Timesheet -> TimesheetEntries (:N) Project -> TimesheetEntries (:N)
Manager -> Employee (1 :N)

1 2 . UI Pages

Authentication - Login Page

Employee - Dashboard - My Timesheets - Add Entry

Manager - Approval Dashboard - Team Timesheets

Admin - Employee Management - Project Management - Holiday Management

Reports - Timesheet Reports

1 3 . Non Functional Requirements

Security - JWT authentication - Role based authorization

Performance - API pagination

Usability - Responsive UI

1 4 . Advanced Features (Recommended)

These features significantly improve usability and make the project stronger for real-world use.

Smart Timesheet Features

- Copy previous day or previous week entries
- Auto-fill common tasks
- Quick add hours using keyboard shortcuts
- Calendar-based timesheet entry view

Timer Based Tracking

Employees can track time using a live timer.

Features: - Start timer - Stop timer - Convert timer data into timesheet entry

1 5 . Holiday and Weekend Handling

System should automatically detect non-working days.

Holiday Rules

- Admin maintains holiday calendar
- Holidays appear automatically in weekly timesheets
- Employees cannot log hours on holidays (configurable)

Weekend Rules

- Saturday and Sunday automatically marked as non-working
 - Optional: allow overtime logging
-

1 6 . Validation Rules

Important rules to prevent incorrect data.

Timesheet rules:

- Maximum 24 hours per day
- Cannot log time on future dates
- Cannot submit empty weekly timesheet
- Cannot edit approved timesheets

Approval rules:

- Manager can only approve team members
-

1 7 . Production Ready Backend Features

To make the system production ready, implement the following backend practices.

API Best Practices

- RESTful API design
- Versioned APIs (/api/v 1 /...)
- Consistent response format

Example:

```
{ "success": true, "data": {}, "message": "Operation successful" }
```

DTO Layer

Use DTOs to separate database entities from API responses.

Example:

- UserDTO
- TimesheetDTO
- TimesheetEntryDTO

Exception Handling

Global exception handler using:

- @ControllerAdvice

Handles:

- Validation errors
- Unauthorized access
- Resource not found

Logging

Use structured logging.

Libraries:

- SLF 4 J
- Logback

Log important events:

- Login attempts
- Timesheet submissions
- Approval actions

API Documentation

Generate API docs automatically using:

- SpringDoc OpenAPI

Provides Swagger UI.

1 8 . Security Features

Security is essential for production systems.

Features:

- JWT authentication
- Password hashing (BCrypt)
- Role based access control
- API authorization filters

Security rules:

Admin: - Full access

Manager: - Access team data

Employee: - Access only personal timesheets

1 9 . Performance Optimization

Important when system scales.

Features:

- API pagination
- Database indexing
- Lazy loading for relations

Optional improvements:

- Redis caching for reports
-

2 0 . Frontend Architecture

Recommended frontend structure.

Folders:

src/

components/ pages/ features/ services/ hooks/ lib/

State Management

Use React Query for:

- Server state
- API caching
- Background refetching

UI Components

Use shadcn/ui for:

- Forms
- Tables
- Dialogs
- Date pickers
- Tabs

UI Features

- Fully responsive UI for mobile, tablet, and desktop screens
- Dark mode support
- Loading skeletons
- Error boundaries
- Form validation
- Mobile friendly navigation
- Responsive tables with horizontal scrolling

Responsive design requirements:

- Mobile breakpoint support
 - Tablet layout optimization
 - Desktop dashboard layout
 - Collapsible sidebar for smaller screens
-

2 1 . Reporting Enhancements

More advanced reporting features.

Reports:

- Employee productivity report
- Monthly hours report
- Project utilization report

Visualization:

- Bar charts
- Weekly hour graphs

Libraries:

- Recharts
-

2 2 . Export and Data Integration

Allow exporting data.

Features:

- Export reports to CSV
- Export reports to Excel

Backend implementation:

- CSV generator
 - Streaming downloads
-

2 3 . Notifications System (Mandatory)

Notifications are a core feature and must be implemented in the system.

Purpose: Improve compliance and ensure employees and managers complete required actions on time.

Types of notifications:

Timesheet reminders - Automatic weekly reminder to submit timesheet - Reminder if the week is ending and the timesheet is still draft

Approval notifications - Manager receives notification when a timesheet is submitted - Employee receives notification when a timesheet is approved - Employee receives notification when a timesheet is rejected

In-app notifications - Notification bell in the UI - Notification list with unread indicators

Backend implementation:

- Notification table
- Event based triggers when actions occur

Fields:

- id
 - user_id
 - message
 - type
 - read_status
 - created_at
-

2 4 . Audit and Activity Tracking

Audit and Activity Tracking

Track important system activities.

Audit logs include:

- Timesheet created
- Timesheet edited
- Timesheet approved
- Timesheet rejected

Fields:

- user
 - action
 - timestamp
-

2 5 . Testing Strategy

Production systems require testing.

Backend testing:

- Unit tests (JUnit)
- Integration tests

Frontend testing:

- Component testing
 - API mocking
-

2 6 . DevOps and Deployment

Recommended deployment setup.

Backend:

- Docker container
- Spring Boot production profile

Frontend:

- Static build deployment

CI/CD pipeline:

- Build
- Run tests
- Deploy

Possible platforms:

- AWS
 - Azure
 - GCP
-

2 7 . Future Advanced Enhancements

If you want to evolve the system further:

- Mobile app
- AI productivity insights
- Integration with payroll systems
- Slack / Teams notifications
- Automatic overtime detection

These enhancements can transform the system into a complete workforce productivity platform.